# Reading Notes for Week 10

## Reinforcement Learning Chapter 1 and 13

### Chapter 1: Introduction

- Markov decision processes
    - The environment in which an agent is placed may subject the agent's actions to being nondeterministic, which means that there are multiple possible successor states that can result from an action taken in some state.
    - Such problems where the world poses a degree of uncertainty are known as nondeterministic search problems, and can be solved with models known as Markov decision processes, or MDPs.
    - Solved using techniques such as value iteration and policy iteration to compute the optimal values of states and extract optimal policies.
        - Value iteration is basically a dynamic programming approach, starting from the ending state and propagating to the starting state.
        - Policy iteration is more computationally efficient and generally faster.
    - Solving Markov decision processes is an example of **offline planning**, where agents have full knowledge of both the transition function and the reward functio and, all the information they need to precompute optimal actions in the world encoded by the MDP without ever actually taking any actions. RL
- Reinforcement Learning
    - Basically **online planning**, during which an agent has no prior knowledge of rewards or transitions in the world (still represented as a MDP). In online planning, an agent must try exploration, during which it performs actions and receives feedback in the form of the successor states it arrives in and the corresponding rewards it reaps.
    - The agent uses this feedback to estimate an optimal policy through a process known as reinforcement learning before using this estimated policy for exploitation or **reward maximization**.
    - In short, reinforcement learning is defined as learning what to do to maximize a numerical reward signal.The learner is not told which actions to take but must discover them through trial-and-error.
- Reinforcement learning is distinguished by its emphasis on an agent learning from direct interaction with its environment without relying on exemplary supervision or complete models of the environment.

### Chapter 13: Policy Gradient Methods

- Policy gradient methods learn a parameterized policy that enables action selection without consulting a value function.

- The policy is differentiable with respect to its parameters, which allows the use of gradient ascent methods in parameter space.
- Policy gradient methods offer advantages over action-value methods, such as the ability to learn stochastic policies and handle continuous action spaces.
    - Also the policy gradient theorem provides a foundation for all policy gradient methods by giving an exact formula for the gradient of performance with respect to the policy parameters.
    - Reinforce, or Monte Carlo Policy Gradient, is introduced as a fundamental algorithm in policy gradient methods, which uses an estimated return following time t as the basis for policy updates.

**Human-level control over deep reinforcement learning**

- The authors basically introduce a new approach to reinforcement learning, which they term Deep Q-Network (DQN). This method enables artificial agents to achieve human-level performance on a wide array of tasks.
    - The DQN combines reinforcement learning with deep neural networks, allowing agents to learn directly from high-dimensional sensory inputs and generalize past experiences to new situations.
    - The architecture of the DQN is inspired by **hierarchical processing system**s in the brain, particularly the visual cortex. It utilizes **deep convolutional neural networks** to process visual input.
- The paper highlights that standard reinforcement learning becomes unstable or may diverge with nonlinear function approximators like neural networks. DQN addresses this by using two key ideas:
    - Experience replay, which randomizes over the data to remove correlations in the observation sequence.
    - An iterative update strategy that adjusts Q-values towards target values that are only periodically updated.
- The DQN was tested on the Atari 2600 gaming platform across 49 games, demonstrating its ability to robustly learn successful policies from visual input alone, without game-specific modifications.
    - From the paper, this method actually outperformed all previous algorithms and reached a performance level comparable to a professional human game tester using the same algorithm, architecture, and hyperparameters across all games.
- Despite the success of DQN, the authors acknowledge that games requiring more temporally extended planning still pose a significant challenge.
- The results illustrate the potential of combining state-of-the-art machine learning techniques with biologically inspired mechanisms to create agents capable of mastering a diverse array of challenging tasks.